Ex. No. 10	Manual User Login Authentication Using Google Firestore Database
Date of Exercise	01 November 2026
Github Code Link(Keep as public)	https://github.com/henrysam/FlutterLaboratory

Aim

To develop a Flutter application that allows users to register and log in using email and password credentials stored in Google Cloud Firestore, and displays a dashboard after successful authentication.

Description

In this experiment, a Flutter application named Secure Login is developed to demonstrate manual user registration and login using Google Cloud Firestore. The application provides registration and login interfaces with email and password input fields. During registration, the entered email address is converted to lowercase and used to generate a document identifier. A random salt is generated, and the password is combined with the salt and hashed using the SHA-256 algorithm provided by the crypto package. The email address, salt, password hash and server timestamp are stored in the Firestore 'users' collection. During login, the application retrieves the corresponding user document, reads the stored salt and password hash, hashes the entered password using the same salt, and compares the result with the stored hash. If the values match, the application navigates to a dashboard displaying the logged-in email address. Otherwise, an appropriate error message is displayed. The interface also supports switching between registration and login, showing or hiding the password, and logging out to return to the login screen. Firebase is initialized using Firebase.initializeApp() with the configuration generated by the FlutterFire CLI. The application was executed in Google Chrome and tested with registration and successful login. This is a classroom demonstration of manual credential checking; production applications should use a dedicated authentication service.

Program

```
import 'dart:convert';
import 'dart:math';

import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:crypto/crypto.dart';
```

```
import 'firebase_options.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );

  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Secure Login',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.indigo,
        ),
        useMaterial3: true,
      ),
      home: const AuthPage(),
    );
  }
}

class AuthPage extends StatefulWidget {
  const AuthPage({super.key});

  @override
  State<AuthPage> createState() => _AuthPageState();
}

class _AuthPageState extends State<AuthPage> {
  final emailController = TextEditingController();
  final passwordController = TextEditingController();

  final users = FirebaseFirestore.instance.collection('users');

  bool isLogin = true;
  bool isLoading = false;
  bool obscurePassword = true;

  String getUserId(String email) {
    return sha256.convert(utf8.encode(email.toLowerCase())).toString();
  }

  String generateSalt() {
    final random = Random.secure();
    final bytes = List<int>.generate(
```

```
    16,
    (_) => random.nextInt(256),
  );
  return base64Encode(bytes);
}

String hashPassword(String password, String salt) {
  return sha256
      .convert(utf8.encode('$salt$password'))
      .toString();
}

Future<void> authenticate() async {
  final email = emailController.text.trim().toLowerCase();
  final password = passwordController.text;

  if (email.isEmpty || password.isEmpty) {
    showMessage('Please enter email and password');
    return;
  }

  if (!email.contains('@') || !email.contains('.')) {
    showMessage('Please enter a valid email address');
    return;
  }

  if (password.length < 6) {
    showMessage('Password must contain at least 6 characters');
    return;
  }

  setState(() => isLoading = true);

  try {
    final userId = getUserId(email);
    final userDoc = await users.doc(userId).get();

    if (isLogin) {
      if (!userDoc.exists) {
        showMessage('Account not found. Please register.');
      } else {
        final data = userDoc.data()!;
        final salt = data['salt'] as String;
        final storedHash = data['passwordHash'] as String;
        final enteredHash = hashPassword(password, salt);

        if (enteredHash == storedHash) {
          if (!mounted) return;

          Navigator.pushReplacement(
            context,
            MaterialPageRoute(
              builder: (_) => DashboardPage(email: email),
            ),
          );
        } else {
          showMessage('Incorrect password');
```

```
      }
     }
    } else {
     if (userDoc.exists) {
      showMessage('Account already exists. Please login.');
     } else {
      final salt = generateSalt();
      final passwordHash = hashPassword(password, salt);

      await users.doc(userId).set({
       'email': email,
       'salt': salt,
       'passwordHash': passwordHash,
       'createdAt': FieldValue.serverTimestamp(),
      });

      showMessage('Registration successful! Please login.');

      setState(() {
       isLogin = true;
       passwordController.clear();
      });
     }
    }
  } catch (e) {
    showMessage('Authentication failed: $e');
  } finally {
    if (mounted) {
     setState(() => isLoading = false);
    }
  }
}

void showMessage(String message) {
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text(message)),
  );
}

@override
void dispose() {
  emailController.dispose();
  passwordController.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: Container(
      width: double.infinity,
      decoration: const BoxDecoration(
        gradient: LinearGradient(
          colors: [
            Color(0xFF3949AB),
            Color(0xFF7986CB),
            Color(0xFFF5F5FF),
          ],
          begin: Alignment.topCenter,
```

```
      end: Alignment.bottomCenter,
    ),
  ),
  child: Center(
    child: SingleChildScrollView(
      padding: const EdgeInsets.all(24),
      child: Card(
        elevation: 8,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(24),
        ),
        child: Padding(
          padding: const EdgeInsets.all(28),
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              const CircleAvatar(
                radius: 35,
                backgroundColor: Color(0xFFE8EAF6),
                child: Icon(
                  Icons.lock_outline,
                  size: 38,
                  color: Colors.indigo,
                ),
              ),
              const SizedBox(height: 20),
              Text(
                isLogin ? 'Welcome Back!' : 'Create Account',
                style: const TextStyle(
                  fontSize: 26,
                  fontWeight: FontWeight.bold,
                ),
              ),
              const SizedBox(height: 8),
              Text(
                isLogin
                    ? 'Login to continue'
                    : 'Register to get started',
                style: TextStyle(color: Colors.grey[600]),
              ),
              const SizedBox(height: 30),
              TextField(
                controller: emailController,
                keyboardType: TextInputType.emailAddress,
                decoration: InputDecoration(
                  labelText: 'Email Address',
                  prefixIcon: const Icon(Icons.email_outlined),
                  border: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(12),
                  ),
                ),
              ),
              const SizedBox(height: 18),
              TextField(
                controller: passwordController,
                obscureText: obscurePassword,
                decoration: InputDecoration(
                  labelText: 'Password',
                  prefixIcon: const Icon(Icons.lock_outline),
                  suffixIcon: IconButton(
                    icon: Icon(
```

```
                obscurePassword
                    ? Icons.visibility_off
                    : Icons.visibility,
              ),
              onPressed: () {
                setState(() {
                  obscurePassword = !obscurePassword;
                });
              },
            ),
            border: OutlineInputBorder(
              borderRadius: BorderRadius.circular(12),
            ),
          ),
        ),
        const SizedBox(height: 25),
        SizedBox(
          width: double.infinity,
          height: 50,
          child: ElevatedButton(
            onPressed: isLoading ? null : authenticate,
            style: ElevatedButton.styleFrom(
              backgroundColor: Colors.indigo,
              foregroundColor: Colors.white,
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(12),
              ),
            ),
            child: isLoading
                ? const CircularProgressIndicator(
                    color: Colors.white,
                  )
                : Text(
                    isLogin ? 'LOGIN' : 'REGISTER',
                    style: const TextStyle(
                      fontWeight: FontWeight.bold,
                    ),
                  ),
          ),
        ),
        const SizedBox(height: 15),
        TextButton(
          onPressed: isLoading
              ? null
              : () {
                  setState(() {
                    isLogin = !isLogin;
                    passwordController.clear();
                  });
                },
          child: Text(
            isLogin
                ? "Don't have an account? Register"
                : 'Already have an account? Login',
          ),
        ),
      ],
    ),
  ),
),
```

```
      ),
    ),
  );
 }
}

class DashboardPage extends StatelessWidget {
 final String email;

 const DashboardPage({super.key, required this.email});

 @override
 Widget build(BuildContext context) {
  return Scaffold(
   appBar: AppBar(
    title: const Text('Dashboard'),
    backgroundColor: Colors.indigo,
    foregroundColor: Colors.white,
    actions: [
     IconButton(
      icon: const Icon(Icons.logout),
      tooltip: 'Logout',
      onPressed: () {
       Navigator.pushReplacement(
        context,
        MaterialPageRoute(
         builder: (_) => const AuthPage(),
        ),
       );
      },
     ),
    ],
   ),
   body: Center(
    child: Padding(
     padding: const EdgeInsets.all(24),
     child: Column(
      mainAxisSize: MainAxisSize.min,
      children: [
       const CircleAvatar(
        radius: 45,
        child: Icon(Icons.person, size: 50),
       ),
       const SizedBox(height: 20),
       const Text(
        'Login Successful!',
        style: TextStyle(
         fontSize: 26,
         fontWeight: FontWeight.bold,
        ),
       ),
       const SizedBox(height: 12),
       Text(
        'Welcome, $email',
        textAlign: TextAlign.center,
        style: const TextStyle(fontSize: 16),
       ),
       const SizedBox(height: 25),
       ElevatedButton.icon(
        onPressed: () {
```

```
                Navigator.pushReplacement(
                  context,
                  MaterialPageRoute(
                    builder: (_) => const AuthPage(),
                  ),
                );
              },
              icon: const Icon(Icons.logout),
              label: const Text('Logout'),
            ),
          ],
        ),
      ),
    ),
  );
 }
}
```

Output Screenshots

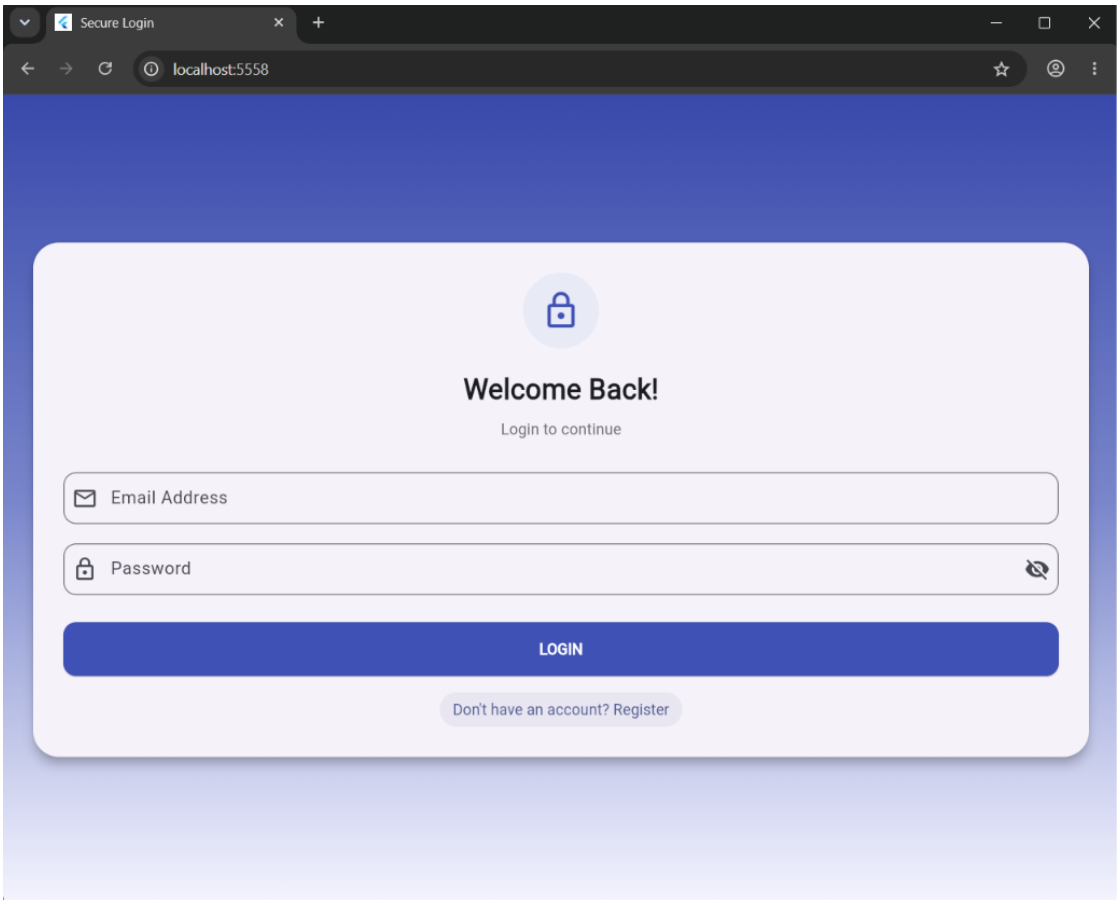


Figure 1: Secure Login application displaying the user login screen.

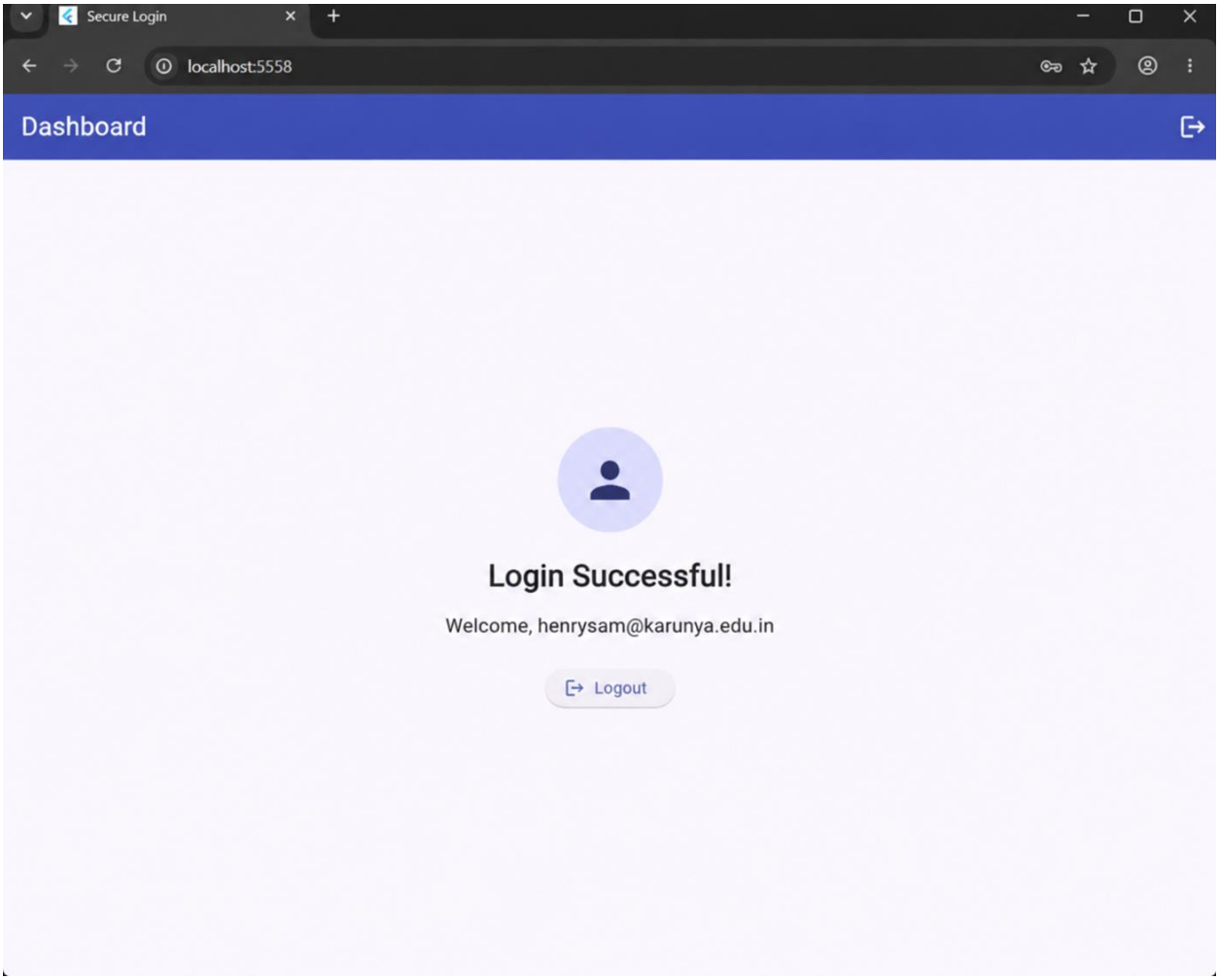


Figure 2: Dashboard displayed after successful user authentication.

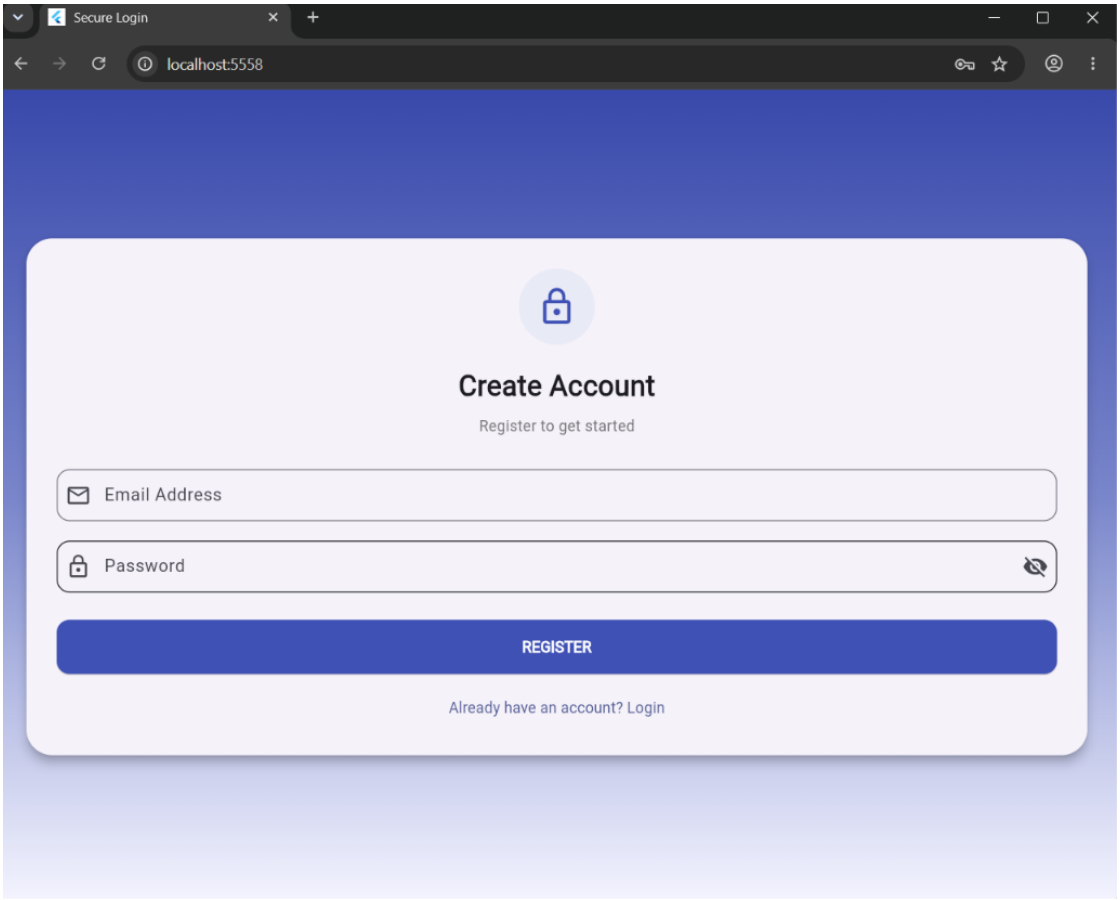


Figure 3: User registration screen for creating a new account.

// Output of the program

Result

Thus, the Flutter application for manual user login authentication using Google Cloud Firestore was successfully developed and executed. The application provided registration and login screens, stored account credentials as salted password hashes in the Firestore 'users' collection, verified the entered password during login, and displayed the dashboard after successful authentication. The registration, login and dashboard screens were tested successfully.